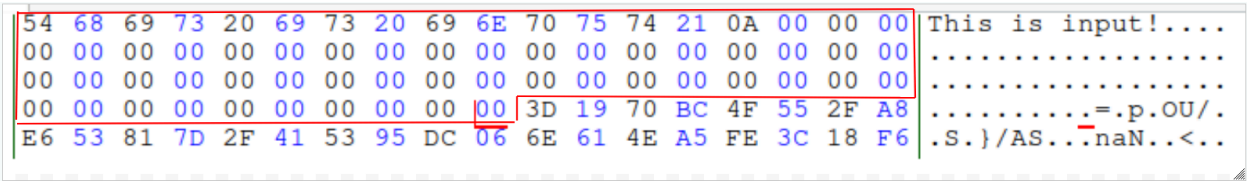
3.1 Task 1: Generating Two Different Files with the Same MD5 Hash

```
[03/13/26]seed@VM:~/A3$ echo "This is input!" > pre.dat
[03/13/26]seed@VM:~/A3$ md5collgen -p pre.dat -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'pre.dat'
Using initial value: 3f0ceba994791808b5d7668503c9bf17

Generating first block: ..........
Generating second block: W.........
Running time: 6.66027 s
[03/13/26]seed@VM:~/A3$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
[03/13/26]seed@VM:~/A3$ md5sum out1.bin
e87fb236ab9d3b4ec6ad9db9505b68cd  out1.bin
[03/13/26]seed@VM:~/A3$ md5sum out2.bin
e87fb236ab9d3b4ec6ad9db9505b68cd  out2.bin
```

1. If the length of the prefix file is not a multiple of 64, md5collision will add filler to reach a byte length divisible by 64. This can also be seen in bless as the padding that appears after the message, but before the collision blocks (64 bytes):

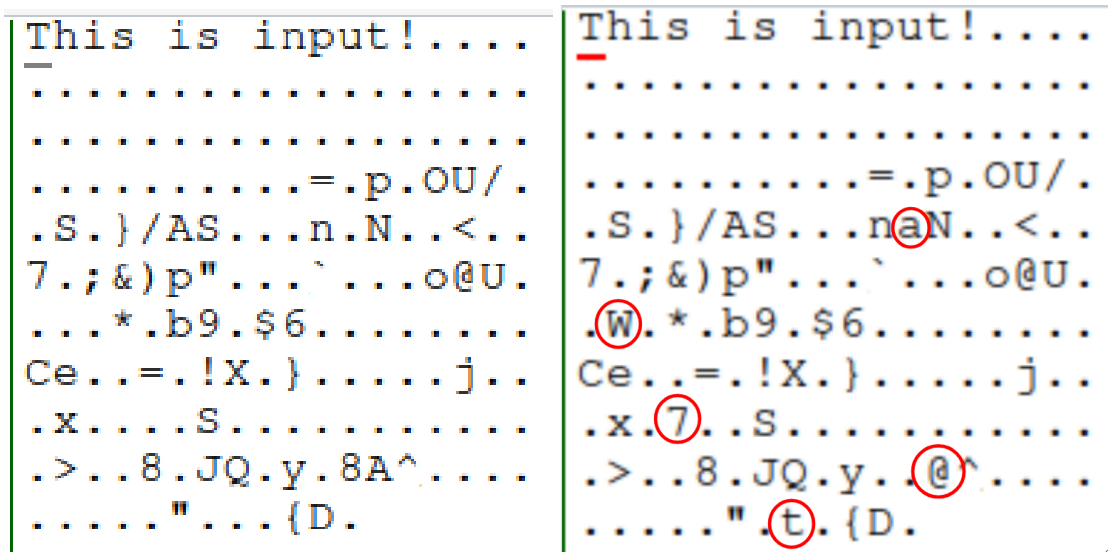


2. If the prefix file is already 64 bytes, no padding would need to be added. This would mean that immediately after the prefix, the collision blocks would begin.

3. No, not all of the 128 bytes added in the collision blocks are identical. The differences can be seen as:

```
[03/13/26]seed@VM:~/A3$ cmp -l out1.bin out2.bin
 84 341 141
110 327 127
111 237 240
124 230  30
148 267  67
174  70 270
175 101 100
188 364 164
```

with 8 byte differences in given byte locations >84. These differences can be seen in the bless UI as:



(Note, only 6 of the 8 are visible here, as the changes in 110/111 and 174/175 are shown as a singular change)

3.2 Task 2: Understanding MD5

```
[03/13/26]seed@VM:~/A3$ echo "extra" > extra.bin
[03/13/26]seed@VM:~/A3$ cat out1.bin extra.bin > out1extra.bin
[03/13/26]seed@VM:~/A3$ cat out2.bin extra.bin > out2extra.bin
[03/13/26]seed@VM:~/A3$ diff out1extra.bin out2extra.bin
Binary files out1extra.bin and out2extra.bin differ
[03/13/26]seed@VM:~/A3$ md5sum out1extra.bin
4b1a9b1ccc53035b9d6628cfc9798b56  out1extra.bin
[03/13/26]seed@VM:~/A3$ md5sum out2extra.bin
4b1a9b1ccc53035b9d6628cfc9798b56  out2extra.bin
```

In this process, I added the word “extra” to the end of each of the output files

output of “bless out1extra.bin”:

```
This is input!.............
..........................
..........=.p.OU/..S.}/AS..
.n.N..<..7.;&)p"...`...o@U.
...*.b9.$6........Ce..=.!X.
}.....j...x....S...........
.>..8.JQ.y.8A^.........."...
{D.extra.
```

Since the property holds that MD5(M ∥ T) = MD5(N ∥ T), the md5sums of the two files still match up. The diff command shows that they still differ, but this is to be expected as the initial changes identified in the previous task carried over.

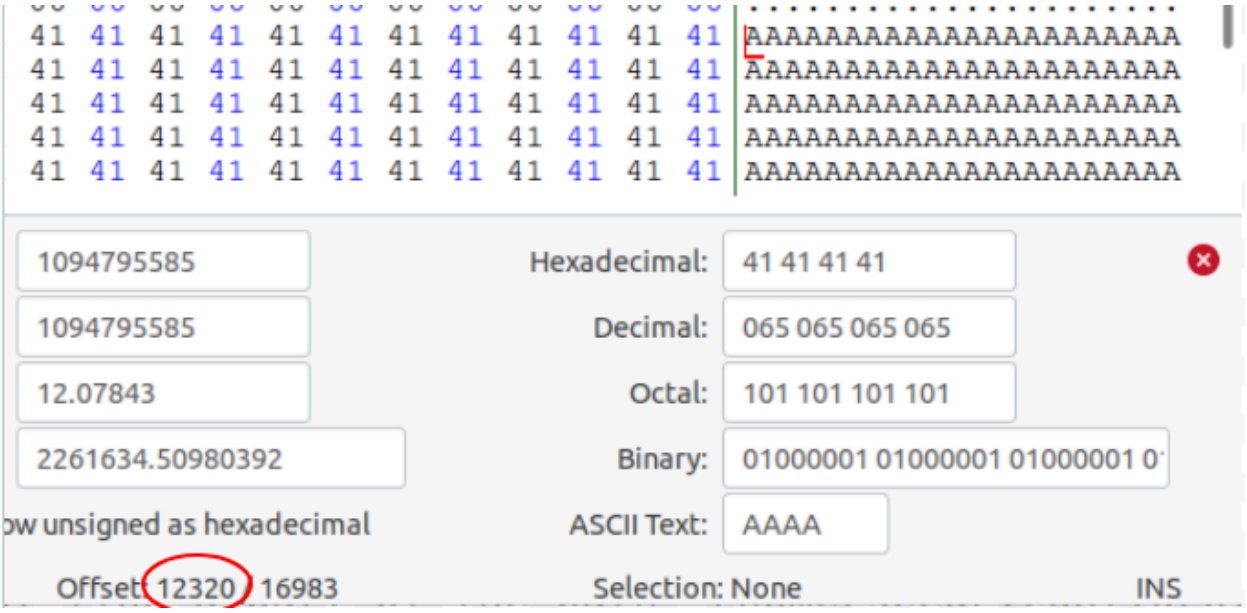
3.3 Task 3: Generating Two Executable Files with the Same MD5 Hash

To begin, I created the C code and compiled it, then went to find the byte value which corresponded to the AAAAAAAAA… array:

```
[03/13/26]seed@VM:~/A3$ cat code.c
#include <stdio.h>
unsigned char xyz[200] = {
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41
};
int main()
{
int i;
for (i=0; i<200; i++){
printf("%x", xyz[i]);
}
printf("\n");
```

```
[03/13/26]seed@VM:~/A3$ gcc code.c -o code
[03/13/26]seed@VM:~/A3$ bless code
```

In the bless UI I was able to find the location for the xyz array, and found that the first value was at offset 12320:



From here, I could begin hashing to create the P and Q values for the prefixes, and create the suffix as well. I used the value 12320 for the head as this was the point to include everything up to (and including) the 12320th byte, which would be the start of the xyz array. I used the value 12449 for the tail end, since the bytes in the range 12321-12449 needed to be removed for the md5collgen collision blocks to exist. The value 12449 was acquired by doing 12320 + 128 (size of collision block) + 1 (since tail -c works starting from not at what would be 12449)

```
[03/13/26]seed@VM:~/A3$ head -c 12320 code > prefix
[03/13/26]seed@VM:~/A3$ md5collgen -p prefix -o P Q
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'P' and 'Q'
Using prefixfile: 'prefix'
Using initial value: 904b5f898347b5a2e7f7dca5fa5ebee0

Generating first block: ..........................
Generating second block: S00.............
Running time: 15.5649 s
[03/13/26]seed@VM:~/A3$ tail -c +12449 code > suffix
```

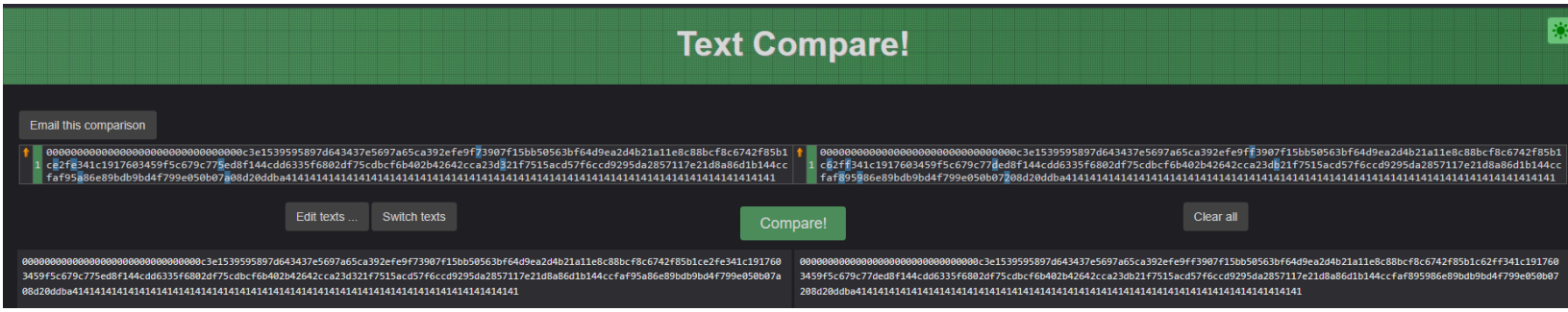
Now that I have my two prefixes Q and P prepared, I can concatenate them with the suffix to produce the two files:

```
[03/13/26]seed@VM:~/A3$ cat P suffix > code1
[03/13/26]seed@VM:~/A3$ cat Q suffix > code2
```

Upon investigating I can see that the two files output different results, but still share the same hash values.

```
[03/13/26]seed@VM:~/A3$ ./code1
00000000000000000000000000000000c3e1539595897d643437e5697a65ca392efe9f73907f15bb50563bf64d9ea2d4b21a11e8c88bcf8
c6742f85b1ce2fe341c1917603459f5c679c775ed8f144cdd6335f6802df75cdbcf6b402b42642cca23d321f7515acd57f6ccd9295da285
7117e21d8a86d1b144ccfaf95a86e89bdb9bd4f799e050b07a08d20ddba4141414141414141414141414141414141414141414141414141
41414141414141414141414141
[03/13/26]seed@VM:~/A3$ ./code2
00000000000000000000000000000000c3e1539595897d643437e5697a65ca392efe9ff3907f15bb50563bf64d9ea2d4b21a11e8c88bcf8
c6742f85b1c62ff341c1917603459f5c679c77ded8f144cdd6335f6802df75cdbcf6b402b42642cca23db21f7515acd57f6ccd9295da285
7117e21d8a86d1b144ccfaf895986e89bdb9bd4f799e050b07208d20ddba4141414141414141414141414141414141414141414141414141
41414141414141414141414141
[03/13/26]seed@VM:~/A3$ md5sum code1 code2
914eb4e4d6a9e5789c418217c4c32b3d  code1
914eb4e4d6a9e5789c418217c4c32b3d  code2
```

This website shows the changes in the text, as it may be hard to spot them.



3.4 Task 4: Making the Two Programs Behave Differently

The idea behind this attack is that two versions of the code will be made (which share the same has value, of course) but will send differing requests to a server for files to download. If the user has control over what files are downloaded from the server, and knows the possible values that can be created from the two different versions, malicious code can be downloaded onto the user's device in one version but not the other. Realistically, external downloads like this probably wouldn't be allowed in an app.

```
[03/13/26]seed@VM:~/A3$ cat code.c
#include <stdio.h>
unsigned char xyz[200] = {
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,
0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41,0x41
};
int main(){
        int size = sizeof(xyz) / sizeof(xyz[0]);
        int sum = 0;
        for (int i = 0; i < size; i++){
                sum += xyz[i];
        }
        printf("Requesting to download from www.AustinSite.com/%d which may or may not be safe!\n", sum);
        }
```

The process is mostly similar to the old one, where 128 bytes from the xyz array are changed, but this time the sum of the xyz array is used. This way, the attacker would know which version of the code would download the benign code, and which the malicious. For instance, www.AustinSite.com/17304 may be benign and safe downloads, where www.AustinSite.com/17305 would be malicious. These values would persist as the copies are distributed.

```
[03/13/26]seed@VM:~/A3$ head -c 12320 code > prefix
[03/13/26]seed@VM:~/A3$ md5collgen -p prefix -o P Q
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'P' and 'Q'
Using prefixfile: 'prefix'
Using initial value: 53ec8a04fa8a57e9daee633a73e17a93

Generating first block: ....
Generating second block: S00.................
Running time: 4.57336 s
[03/13/26]seed@VM:~/A3$ tail -c +12449 code > suffix
[03/13/26]seed@VM:~/A3$ cat P suffix > code1
[03/13/26]seed@VM:~/A3$ cat Q suffix > code2
[03/13/26]seed@VM:~/A3$ chmod 777 code1 code2
[03/13/26]seed@VM:~/A3$ ./code1
Requesting to download from www.AustinSite.com/17304 which may or may not be safe!
[03/13/26]seed@VM:~/A3$ ./code2
Requesting to download from www.AustinSite.com/17305 which may or may not be safe!
```